

1. Deep-Dive Mathematical Derivation from First Principles

In the context of a Central Risk Book (CRB) position at Nomura, understanding latent regimes is critical. When modeling equity market returns, parameters like volatility and correlation drift over time. This makes a static model highly problematic for capital allocation.

The Hidden Markov Model (HMM) parameterizes these latent regimes as discrete unobserved states $s_t \in \{1, \dots, K\}$. The model is fully defined by the tuple $\theta = (\mathbf{A}, \mathbf{B}, \boldsymbol{\pi})$, where:

- $\mathbf{A} \in \mathbb{R}^{K \times K}$ is the transition matrix with components $A_{ij} = \mathbb{P}(s_t = j \mid s_{t-1} = i)$.
- $\mathbf{B}$ is the set of emission parameters. For continuous equity returns $r_t \in \mathbb{R}$, we specify a Gaussian emission probability distribution: $b_j(r_t) = p(r_t \mid s_t = j) = \frac{1}{\sqrt{2\pi\sigma_j^2}} \exp\left(-\frac{(r_t - \mu_j)^2}{2\sigma_j^2}\right)$.
- $\boldsymbol{\pi} \in \mathbb{R}^K$ is the initial state probability distribution vector where $\pi_j = \mathbb{P}(s_1 = j)$.

Because the state sequence $S = \{s_1, \dots, s_T\}$ is hidden, the direct likelihood function is a mixture over K^T states, which is computationally intractable ($O(K^T)$). To optimize this, the **Baum-Welch algorithm** utilizes an Expectation-Maximization (EM) framework to maximize the log-likelihood by iteratively evaluating the forward-backward variables.

Phase A: Forward Recursion Derivation

The forward variable $\alpha_t(j)$ is defined as the joint probability of observing the partial return sequence up to time t and terminating in hidden state j at time t , given model parameters θ :

$$\alpha_t(j) = \mathbb{P}(r_1, r_2, \dots, r_t, s_t = j \mid \theta)$$

Step 1: Base Case Initialization ($t = 1$)

By the definition of conditional probability:

$$\alpha_1(j) = \mathbb{P}(r_1, s_1 = j \mid \theta) = \mathbb{P}(s_1 = j \mid \theta) \cdot \mathbb{P}(r_1 \mid s_1 = j, \theta)$$

Substituting our structural parameters π_j and $b_j(r_1)$:

$$\alpha_1(j) = \pi_j b_j(r_1)$$

Step 2: Inductive Step ($t - 1 \rightarrow t$)

We express $\alpha_t(j)$ by marginalizing over all possible hidden states i at time $t - 1$:

$$\alpha_t(j) = \sum_{i=1}^K \mathbb{P}(r_1, \dots, r_{t-1}, r_t, s_{t-1} = i, s_t = j \mid \theta)$$

Apply the product rule to isolate the current observation r_t :

$$\alpha_t(j) = \sum_{i=1}^K \mathbb{P}(r_t \mid r_1, \dots, r_{t-1}, s_{t-1} = i, s_t = j, \theta) \cdot \mathbb{P}(r_1, \dots, r_{t-1}, s_{t-1} = i, s_t = j \mid \theta)$$

By the Markov property of HMMs, the emission r_t depends solely on the current state $s_t = j$:

$$\mathbb{P}(r_t \mid r_1, \dots, r_{t-1}, s_{t-1} = i, s_t = j, \theta) = \mathbb{P}(r_t \mid s_t = j, \theta) = b_j(r_t)$$

Now factor the remaining joint probability term:

$$\mathbb{P}(r_1, \dots, r_{t-1}, s_{t-1} = i, s_t = j \mid \theta) = \mathbb{P}(s_t = j \mid r_1, \dots, r_{t-1}, s_{t-1} = i, \theta) \cdot \mathbb{P}(r_1, \dots, r_{t-1}, s_{t-1} = i \mid \theta)$$

Since state transitions depend only on the immediate prior state s_{t-1} :

$$\mathbb{P}(s_t = j \mid r_1, \dots, r_{t-1}, s_{t-1} = i, \theta) = \mathbb{P}(s_t = j \mid s_{t-1} = i) = A_{ij}$$

Recognizing that the final term is the definition of the forward variable at time $t - 1$, $\alpha_{t-1}(i) = \mathbb{P}(r_1, \dots, r_{t-1}, s_{t-1} = i \mid \theta)$, we substitute everything back to yield the recursive relationship:

$$\alpha_t(j) = \sum_{i=1}^K b_j(r_t) \cdot A_{ij} \cdot \alpha_{t-1}(i) = b_j(r_t) \sum_{i=1}^K \alpha_{t-1}(i) A_{ij}$$

Phase B: Backward Recursion Derivation

The backward variable $\beta_t(j)$ is the conditional probability of observing the remaining return sequence from time $t + 1$ to T , given that the model is in hidden state j at time t :

$$\beta_t(j) = \mathbb{P}(r_{t+1}, r_{t+2}, \dots, r_T \mid s_t = j, \theta)$$

Step 1: Base Case Initialization ($t = T$)

By definition, there are no subsequent observations after time T . Thus, the probability of observing an empty set of future returns is identically 1:

$$\beta_T(j) = 1$$

Step 2: Inductive Step ($t + 1 \rightarrow t$)

We express $\beta_t(j)$ by marginalizing over all possible states k at time $t + 1$:

$$\beta_t(j) = \sum_{k=1}^K \mathbb{P}(r_{t+1}, r_{t+2}, \dots, r_T, s_{t+1} = k \mid s_t = j, \theta)$$

Decompose the joint term into conditional components using the product rule:

$$\beta_t(j) = \sum_{k=1}^K \mathbb{P}(r_{t+2}, \dots, r_T \mid s_t = j, s_{t+1} = k, r_{t+1}, \theta) \cdot \mathbb{P}(r_{t+1}, s_{t+1} = k \mid s_t = j, \theta)$$

Using the conditional independence properties of the Markov chain, future returns $r_{t+2} \dots r_T$ depend only on the state at time $t + 1$:

$$\mathbb{P}(r_{t+2}, \dots, r_T \mid s_t = j, s_{t+1} = k, r_{t+1}, \theta) = \mathbb{P}(r_{t+2}, \dots, r_T \mid s_{t+1} = k, \theta) = \beta_{t+1}(k)$$

Now decompose the second conditional probability component:

$$\mathbb{P}(r_{t+1}, s_{t+1} = k \mid s_t = j, \theta) = \mathbb{P}(r_{t+1} \mid s_{t+1} = k, s_t = j, \theta) \cdot \mathbb{P}(s_{t+1} = k \mid s_t = j, \theta)$$

The return r_{t+1} depends entirely on its contemporaneous state $s_{t+1} = k$, simplifying the first term to $b_k(r_{t+1})$. The second term matches the definition of the state transition probability A_{jk} . Combining these gives the backward recursive equation:

$$\beta_t(j) = \sum_{k=1}^K \beta_{t+1}(k) \cdot b_k(r_{t+1}) \cdot A_{jk} = \sum_{k=1}^K A_{jk} b_k(r_{t+1}) \beta_{t+1}(k)$$

Phase C: Expectation (E) Step

During the E-step, we use the forward and backward variables calculated under the current parameters θ to compute the latent state probabilities (responsibilities).

Step 1: Single-State Responsibility $\gamma_t(j)$

$\gamma_t(j)$ is the conditional probability that the model is in state j at time t , given the entire observed sequence $r_{1:T}$:

$$\gamma_t(j) = \mathbb{P}(s_t = j \mid r_{1:T}, \theta) = \frac{\mathbb{P}(r_{1:T}, s_t = j \mid \theta)}{\mathbb{P}(r_{1:T} \mid \theta)}$$

Split the full observation sequence into past-to-present $r_{1:t}$ and future $r_{t+1:T}$ blocks:

$$\mathbb{P}(r_{1:T}, s_t = j \mid \theta) = \mathbb{P}(r_{1:t}, r_{t+1:T}, s_t = j \mid \theta) = \mathbb{P}(r_{t+1:T} \mid r_{1:t}, s_t = j, \theta) \cdot \mathbb{P}(r_{1:t}, s_t = j \mid \theta)$$

Due to conditional independence, future emissions do not depend on past emissions when conditioned on the current state:

$$\mathbb{P}(r_{t+1:T} \mid r_{1:t}, s_t = j, \theta) = \mathbb{P}(r_{t+1:T} \mid s_t = j, \theta) = \beta_t(j)$$

The second term is the forward variable $\alpha_t(j)$. Thus, the numerator simplifies to:

$$\mathbb{P}(r_{1:T}, s_t = j \mid \theta) = \alpha_t(j) \beta_t(j)$$

The denominator $\mathbb{P}(r_{1:T} \mid \theta)$ represents the total likelihood of the sequence, found by marginalizing over all states at time t :

$$\mathbb{P}(r_{1:T} \mid \theta) = \sum_{k=1}^K \mathbb{P}(r_{1:T}, s_t = k \mid \theta) = \sum_{k=1}^K \alpha_t(k) \beta_t(k)$$

Combining these yields the expression for single-state responsibility:

$$\gamma_t(j) = \frac{\alpha_t(j) \beta_t(j)}{\sum_{k=1}^K \alpha_t(k) \beta_t(k)}$$

Step 2: Transition Responsibility $\xi_t(i, j)$

$\xi_t(i, j)$ is the joint probability of transitioning from state i at time t to state j at time $t + 1$, given the complete return stream:

$$\xi_t(i, j) = \mathbb{P}(s_t = i, s_{t+1} = j \mid r_{1:T}, \theta) = \frac{\mathbb{P}(r_{1:T}, s_t = i, s_{t+1} = j \mid \theta)}{\mathbb{P}(r_{1:T} \mid \theta)}$$

Decompose the numerator by breaking the time series at index t and $t + 1$:

$$\mathbb{P}(r_{1:T}, s_t = i, s_{t+1} = j \mid \theta) = \mathbb{P}(r_{1:t}, s_t = i) \cdot \mathbb{P}(s_{t+1} = j \mid s_t = i) \cdot \mathbb{P}(r_{t+1} \mid s_{t+1} = j) \cdot \mathbb{P}(r_{t+2:T} \mid s_{t+1} = j)$$

Map each component to its corresponding HMM term:

- $\mathbb{P}(r_{1:t}, s_t = i) = \alpha_t(i)$

- $\mathbb{P}(s_{t+1} = j \mid s_t = i) = A_{ij}$
- $\mathbb{P}(r_{t+1} \mid s_{t+1} = j) = b_j(r_{t+1})$
- $\mathbb{P}(r_{t+2:T} \mid s_{t+1} = j) = \beta_{t+1}(j)$

Substitute these back into the expression, using the marginalized denominator:

$$\xi_t(i, j) = \frac{\alpha_t(i) A_{ij} b_j(r_{t+1}) \beta_{t+1}(j)}{\sum_{k=1}^K \sum_{l=1}^K \alpha_t(k) A_{kl} b_l(r_{t+1}) \beta_{t+1}(l)}$$

Phase D: Maximization (M) Step

The M-step updates the parameter set $\hat{\theta} = (\hat{\mathbf{A}}, \hat{\mathbf{B}}, \hat{\boldsymbol{\pi}})$ by maximizing the expected log-likelihood found in the E-step.

Step 1: Initial State Probability Update $\hat{\pi}_j$

The initial state assignment matches the model's responsibility profile at index $t = 1$:

$$\hat{\pi}_j = \gamma_1(j)$$

Step 2: Transition Matrix Update $\hat{A}_{ij}$

The updated entry $\hat{A}_{ij}$ is the ratio of the expected number of transitions from state i to state j to the total expected number of transitions out of state i :

$$\hat{A}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

Step 3: Gaussian Emission Parameter Updates $(\hat{\mu}_j, \hat{\sigma}_j^2)$

To derive the continuous emission updates, we formulate the Lagrangian with respect to the continuous density functions weighted by the state responsibility $\gamma_t(j)$.

For the mean vector $\hat{\mu}_j$, we set the partial derivative of the expected log-likelihood to zero:

$$\begin{aligned} \frac{\partial}{\partial \mu_j} \sum_{t=1}^T \gamma_t(j) \ln \left(\frac{1}{\sqrt{2\pi\sigma_j^2}} \exp \left(-\frac{(r_t - \mu_j)^2}{2\sigma_j^2} \right) \right) &= 0 \\ \frac{\partial}{\partial \mu_j} \sum_{t=1}^T \gamma_t(j) \left[-\frac{1}{2} \ln(2\pi\sigma_j^2) - \frac{(r_t - \mu_j)^2}{2\sigma_j^2} \right] &= 0 \\ \sum_{t=1}^T \gamma_t(j) \left[\frac{2(r_t - \mu_j)}{2\sigma_j^2} \right] &= 0 \implies \frac{1}{\sigma_j^2} \sum_{t=1}^T \gamma_t(j) (r_t - \mu_j) = 0 \\ \sum_{t=1}^T \gamma_t(j) r_t - \mu_j \sum_{t=1}^T \gamma_t(j) &= 0 \implies \hat{\mu}_j = \frac{\sum_{t=1}^T \gamma_t(j) r_t}{\sum_{t=1}^T \gamma_t(j)} \end{aligned}$$

Similarly, optimizing for the variance parameter $\hat{\sigma}_j^2$:

$$\frac{\partial}{\partial \sigma_j^2} \sum_{t=1}^T \gamma_t(j) \left[-\frac{1}{2} \ln(2\pi) - \frac{1}{2} \ln(\sigma_j^2) - \frac{(r_t - \mu_j)^2}{2\sigma_j^2} \right] = 0$$

$$\sum_{t=1}^T \gamma_t(j) \left[-\frac{1}{2\sigma_j^2} + \frac{(r_t - \mu_j)^2}{2(\sigma_j^2)^2} \right] = 0$$

Multiply the entire equation by $2(\sigma_j^2)^2$:

$$\sum_{t=1}^T \gamma_t(j) [-\sigma_j^2 + (r_t - \mu_j)^2] = 0 \implies -\sigma_j^2 \sum_{t=1}^T \gamma_t(j) + \sum_{t=1}^T \gamma_t(j)(r_t - \mu_j)^2 = 0$$

$$\hat{\sigma}_j^2 = \frac{\sum_{t=1}^T \gamma_t(j)(r_t - \hat{\mu}_j)^2}{\sum_{t=1}^T \gamma_t(j)}$$

2. Quantitative Infrastructure & Application of Theory

The following production-grade Python script implements a 3-State Hidden Markov Model from scratch using numpy and scipy. It avoids external high-level wrappers to demonstrate an exact programmatic translation of the forward-backward equations.

To prevent numerical underflow over long sequences, calculations are performed using log-space transformations or local scaling factor normalization.

```
import numpy as np
import pandas as pd
from scipy.stats import norm
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# =====
# 1. SYNTHETIC MULTI-REGIME RETURN GENERATOR
# =====
def generate_regime_switching_data(n_samples: int = 1500, seed: int = 42) -> tuple:
    """
    Generates time-series returns driven by a latent 3-state Markov chain.
    Regimes represent: 0 = Bull (Low Vol), 1 = Neutral, 2 = Crisis (High Vol).
    """
    np.random.seed(seed)

    # Structural Ground Truth Parameters
    true_pi = np.array([0.6, 0.3, 0.1])
    true_A = np.array([
        [0.95, 0.04, 0.01], # From Bull
        [0.05, 0.90, 0.05], # From Neutral
        [0.02, 0.18, 0.80]  # From Crisis
    ])
    true_mu = np.array([0.0005, 0.0001, -0.0025])
    true_sigma = np.array([0.006, 0.012, 0.035])

    states = np.zeros(n_samples, dtype=int)
    returns = np.zeros(n_samples)

    # Initialize initial state
    states = np.random.choice(3, p=true_pi)
    returns = np.random.normal(loc=true_mu[states], scale=true_sigma[states])
```

```

# Generate path trajectories
for t in range(1, n_samples):
    current_state = states[t-1]
    next_state = np.random.choice(, p=true_A[current_state, :])
    states[t] = next_state
    returns[t] = np.random.normal(loc=true_mu[next_state],
scale=true_sigma[next_state])

    return returns, states, (true_A, true_mu, true_sigma)

# =====
# 2. BAUM-WELCH OPTIMIZATION ENGINE FROM FIRST PRINCIPLES
# =====
class ProductionGaussianHMM:
    """
    3-State Hidden Markov Model for continuous returns parameter estimation.
    Uses local scaling factors (c_t) to prevent numerical floating-point underflow.
    """

    def __init__(self, n_states: int = 3, max_iter: int = 100, tol: float = 1e-6):
        self.K = n_states
        self.max_iter = max_iter
        self.tol = tol

        # Parameters to fit
        self.A = None
        self.mu = None
        self.sigma = None
        self.pi = None

    def _initialize_parameters(self, returns: np.ndarray):
        """K-means type heuristic initialization of states."""
        self.pi = np.ones(self.K) / self.K
        # Uniform transition matrix initialization with strong diagonal permanence
        self.A = np.full((self.K, self.K), 0.1 / (self.K - 1))
        np.fill_diagonal(self.A, 0.9)

        # Split data chunks to seed distinct means and variances
        chunks = np.array_split(np.sort(returns), self.K)
        self.mu = np.array([np.mean(c) for c in chunks])
        self.sigma = np.array([np.std(c) for c in chunks])

    def fit(self, r: np.ndarray):
        self._initialize_parameters(r)
        T = len(r)

        old_log_likelihood = -np.inf

        for iteration in range(self.max_iter):
            # -----
            # EXPECTATION STEP (FORWARD-BACKWARD RECURSIONS WITH SCALING)
            # -----
            alpha = np.zeros((T, self.K))
            beta = np.zeros((T, self.K))
            c = np.zeros(T) # Underflow scaling factor array

            # Compute emissions matrix: B[t, j] = b_j(r_t)
            B = np.zeros((T, self.K))

```

```

for j in range(self.K):
    B[:, j] = norm.pdf(r, loc=self.mu[j], scale=self.sigma[j])
B = np.clip(B, 1e-12, None) # Guard against zero values

# Forward Base Initialization
alpha[0, :] = self.pi * B[0, :]
c = 1.0 / np.sum(alpha[0, :])
alpha[0, :] *= c

# Forward Induction Loop
for t in range(1, T):
    alpha[t, :] = (alpha[t-1, :] @ self.A) * B[t, :]
    c[t] = 1.0 / np.sum(alpha[t, :])
    alpha[t, :] *= c[t]

# Backward Base Initialization
beta[T-1, :] = 1.0 * c[T-1]

# Backward Induction Loop
for t in range(T-2, -1, -1):
    beta[t, :] = (self.A @ (B[t+1, :] * beta[t+1, :])) * c[t]

# Compute Latent Responsibilities (Gamma & Xi)
# Reconstruct unscaled denominator product identity
gamma = alpha * beta
gamma = gamma / np.sum(gamma, axis=1, keepdims=True)

xi = np.zeros((T-1, self.K, self.K))
for t in range(T-1):
    numerator = np.outer(alpha[t, :], B[t+1, :] * beta[t+1, :]) * self.A
    xi[t, :, :] = numerator / np.sum(numerator)

# Compute exact Log-Likelihood from scaling variables
log_likelihood = -np.sum(np.log(c))

# Convergence Criteria Evaluation
if abs(log_likelihood - old_log_likelihood) < self.tol:
    break
old_log_likelihood = log_likelihood

# -----
# MAXIMIZATION STEP (PARAMETER UPDATES)
# -----
self.pi = gamma[0, :] / np.sum(gamma[0, :])

# Transition Matrix Updates
self.A = np.sum(xi, axis=0) / np.sum(gamma[:-1, :], axis=0)[:, np.newaxis]

# Emission Parameters Vector Updates
for j in range(self.K):
    sum_gamma = np.sum(gamma[:, j])
    self.mu[j] = np.sum(gamma[:, j] * r) / sum_gamma
    self.sigma[j] = np.sqrt(np.sum(gamma[:, j] * (r - self.mu[j])**2) /
sum_gamma)

return self, log_likelihood

```

```

def predict_responsibilities(self, r: np.ndarray) -> np.ndarray:
    """Returns the smooth gamma probability spectrum across states."""
    T = len(r)
    alpha = np.zeros((T, self.K))
    c = np.zeros(T)
    B = np.zeros((T, self.K))
    for j in range(self.K): B[:, j] = norm.pdf(r, loc=self.mu[j],
scale=self.sigma[j])

    alpha[0, :] = self.pi * B[0, :]
    c = 1.0 / np.sum(alpha[0, :])
    alpha[0, :] *= c

    for t in range(1, T):
        alpha[t, :] = (alpha[t-1, :] @ self.A) * B[t, :]
        c[t] = 1.0 / np.sum(alpha[t, :])
        alpha[t, :] *= c[t]

    beta = np.zeros((T, self.K))
    beta[T-1, :] = 1.0 * c[T-1]
    for t in range(T-2, -1, -1):
        beta[t, :] = (self.A @ (B[t+1, :] * beta[t+1, :])) * c[t]

    gamma = alpha * beta
    return gamma / np.sum(gamma, axis=1, keepdims=True)

# =====
# 3. INTERACTIVE PRODUCTION PLOTLY DASHBOARD GENERATOR
# =====
def generate_hmm_dashboard(returns: np.ndarray, true_states: np.ndarray, gamma_probs:
np.ndarray, bic_scores: dict):
    """
    Renders an interactive multi-panel dashboard illustrating returns tracking,
    latent probability assignments, and structural criteria scores.
    """
    fig = make_subplots(
        rows=3, cols=1,
        shared_xaxes=False,
        vertical_spacing=0.08,
        subplot_titles=(
            "Observed Asset Returns Contextualized with True Latent States",
            "Inferred Latent State Responsibility Spectrum (Smooth Gamma Filter)",
            "Model Selection Topology via Bayesian Information Criterion (BIC)"
        )
    )

    # Panel 1: Returns colored by ground truth state
    time_axis = np.arange(len(returns))
    colors = {0: 'rgba(46, 204, 113, 0.8)', 1: 'rgba(241, 196, 15, 0.8)', 2: 'rgba(231,
76, 60, 0.8)'}
    state_labels = {0: 'Bull (Low Vol)', 1: 'Neutral', 2: 'Crisis (High Vol)'}

    for s_idx in range(3):
        mask = (true_states == s_idx)
        fig.add_trace(go.Scatter(
            x=time_axis[mask], y=returns[mask], mode='markers',
            marker=dict(color=colors[s_idx], size=4),

```

```

        name=state_labels[s_idx], legendgroup="group1"
    ), row=1, col=1)

    # Panel 2: Responsibility spectrum curves
    fig.add_trace(go.Scatter(x=time_axis, y=gamma_probs[:, 0], mode='lines',
line=dict(color='emerald', width=1), name='P(State 0) Prob', legendgroup="group2"),
row=2, col=1)
    fig.add_trace(go.Scatter(x=time_axis, y=gamma_probs[:, 1], mode='lines',
line=dict(color='gold', width=1), name='P(State 1) Prob', legendgroup="group2"), row=2,
col=1)
    fig.add_trace(go.Scatter(x=time_axis, y=gamma_probs[:, 2], mode='lines',
line=dict(color='crimson', width=1), name='P(State 2) Prob', legendgroup="group2"),
row=2, col=1)

    # Panel 3: BIC Model Selection Analysis
    fig.add_trace(go.Scatter(
        x=list(bic_scores.keys()), y=list(bic_scores.values()),
        mode='lines+markers', marker=dict(size=8, color='cyan'),
        line=dict(width=3, dash='solid'), name='BIC Path Curve'
    ), row=3, col=1)

    # Layout Adjustments
    fig.update_layout(
        title_text="Nomura Cash Equities Quantitative Research – Latent Market Regime
Processing Module",
        title_x=0.5, height=950, width=1250, template="plotly_dark", showlegend=True
    )

    fig.update_yaxes(title_text="Returns Profile", row=1, col=1)
    fig.update_yaxes(title_text="Responsibility Prob", row=2, col=1)
    fig.update_xaxes(title_text="Number of Hidden States K", row=3, col=1)
    fig.update_yaxes(title_text="BIC Score (Lower is Optimal)", row=3, col=1)

    fig.show()

if __name__ == "__main__":
    # 1. Pipe underlying synthetic return profile
    sim_returns, sim_states, truth_tuple =
generate_regime_switching_data(n_samples=1200)

    # 2. Iterate through cross-validation configurations to determine state selection
parameters
    bic_metrics = {}
    for test_k in:
        test_hmm = ProductionGaussianHMM(n_states=test_k, max_iter=60)
        _, final_ll = test_hmm.fit(sim_returns)

        # Calculate free parameters: Initial vector (K-1) + Transition Rows (K*(K-1)) +
Gaussian Means (K) + Volatilities (K)
        num_free_parameters = (test_k - 1) + (test_k * (test_k - 1)) + test_k + test_k
        bic_metrics[test_k] = -2 * final_ll + num_free_parameters *
np.log(len(sim_returns))

    # 3. Fit optimal production model (K=3)
    production_hmm = ProductionGaussianHMM(n_states=3, max_iter=100)
    production_hmm, _ = production_hmm.fit(sim_returns)
    inferred_gammas = production_hmm.predict_responsibilities(sim_returns)

```

```

print("\n" + "="*60)
print("ESTIMATED GAUSSIAN HMM PARAMETER REGIMES:")
print("="*60)
for k in range(3):
    print(f"Regime State {k} -> Inferred Mean: {production_hmm.mu[k]:.5f} |
Inferred Vol: {production_hmm.sigma[k]:.5f}")
print("\nInferred Transition Matrix A:")
print(production_hmm.A)
print("="*60 + "\n")

# 4. Launch visual platform
generate_hmm_dashboard(sim_returns, sim_states, inferred_gammas, bic_metrics)

```

3. Strategic Interview Follow-Ups & Edge-Case Vulnerabilities

Follow-Up 1: Local vs Global Optima in Baum-Welch (EM)

Question: "The Baum-Welch algorithm is a local hill-climbing optimization algorithm. If your initialization parameters θ_0 are poorly chosen, it will converge to an suboptimal local maximum. In our CRB context, what concrete initialization strategy do you apply to avoid this?"

- **The Answer:** Relying on uniform or random initialization is a major risk for production systems. To ensure convergence to a global optimum, we use a two-tiered initialization framework:
 1. **Averaging Partitioning via Structural Constraints:** We map the data to empirical proxies. We run a rolling empirical window volatility estimator (e.g., 20-day realized variance) across the asset returns.
 2. **Deterministic Clustering (K-Means+):** We segment the observed returns based on this volatility metric into K discrete subsets. We compute the empirical mean vector μ_{init} and variance vector σ_{init}^2 directly from these clustered blocks to form our initialization parameters $\mathbf{B}_0$.
 3. **Transition Priors:** We initialize the transition matrix $\mathbf{A}_0$ with strong diagonal permanence (e.g., 0.95 on the diagonal) to reflect the empirical reality that financial market regimes are highly persistent.

Follow-Up 2: The Non-Stationarity of the Transition Matrix $\mathbf{A}$

Question: "Your model assumes the state transition matrix A_{ij} is constant over time (homogeneous Markov chain). In reality, during financial contagion events, transition probabilities shift instantly as liquidity dries up. How do you extend this setup to handle time-varying transition dynamics?"

- **The Answer:** We must transition from a Homogeneous HMM to a **Non-Homogeneous Hidden Markov Model (NHMM)**. Instead of treating A_{ij} as a fixed scalar coefficient, we parameterize the transition probability as a logistic function of exogenous time-varying market microstructural features, denoted as $\mathbf{x}_t$ (such as changes in market-wide VIX, or the Central Risk Book's aggregate risk exposure):

$$A_{ij}(t) = \mathbb{P}(s_t = j \mid s_{t-1} = i, \mathbf{x}_t) = \frac{\exp(\mathbf{w}_{ij}^T \mathbf{x}_t)}{\sum_{l=1}^K \exp(\mathbf{w}_{il}^T \mathbf{x}_t)}$$

During the optimization loop, the M-step can no longer be evaluated analytically via a simple ratio summary ($\frac{\sum \xi}{\gamma}$). Instead, we update the transition weight parameters $\mathbf{w}_{ij}$ numerically using a gradient-based optimization step (such

as L-BFGS) within the M-step loop. This allows the model to capture how macroeconomic shocks or structural order book shifts dynamically alter regime transition frequencies.